

SwipeGen: Bridging the Execution Gap in GUI Agents via Human-like Swipe Synthesis

Xuan Wang*
College of Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
xuanwang25@m.fudan.edu.cn

Siyuan Su*
College of Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
24300810015@m.fudan.edu.cn

Quantong Fu*
College of Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
qtfu22@m.fudan.edu.cn

Yongxiang Hu
College of Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
huyx17@fudan.edu.cn

Yangfan Zhou[✉]
College of Computer Science and
Artificial Intelligence
Fudan University
Shanghai, China
zyf@fudan.edu.cn

Abstract

Despite numerous Graphical User Interface (GUI) agents claiming to automate user interaction tasks, to date, few achieve satisfactory interaction capability with human users in real-world scenarios. Through empirical analysis, this paper identifies the root cause of the limited interaction capability as the rigid swipe execution. In particular, unlike humans, who perform swipes with fine-grained control over trajectory, speed, and timing, existing agents can only conduct simplistic, deterministic swipe behaviors, leading to frequent failures on complicated user-like interaction tasks. Due to the lack of open-source human-like swipe training data, we propose SwipeGen, the first tool for synthesizing diverse and human-like swipe interactions, and SwipeBench, the first benchmark for evaluating agents' swipe interaction quality. Extensive experiments show that SwipeGen can improve the swipe execution success rate of existing agents by up to 2.46X. Our code, dataset, and model are available at <https://github.com/TSKGHS17/SwipeGen>.

CCS Concepts

• **Software and its engineering** → **Software system models**; • **Human-centered computing** → **Interaction techniques**.

Keywords

GUI agents, Swipe execution, Automated data synthesis, Vision-language models

ACM Reference Format:

Xuan Wang, Siyuan Su, Quantong Fu, Yongxiang Hu, and Yangfan Zhou. 2026. SwipeGen: Bridging the Execution Gap in GUI Agents via Human-like Swipe Synthesis. In *Proceedings of the 34th ACM International Conference on*

Multimedia (MM '26), November 10–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3767308.3835803>

1 Introduction

Graphical User Interface (GUI) agents [8, 27] are autonomous systems that interact with GUIs in a human-like manner based on natural language commands. These agents have found applications in various domains, such as mobile assistants [35, 36, 44], accessibility tools [17, 29], and automated GUI testing [13, 16, 23, 38]. Recently, with the rise of Vision-Language Models (VLMs), VLM-based GUI agents [9, 14, 15, 21, 25, 26, 39] have become increasingly popular. This is because VLMs can process both visual (*e.g.*, GUI images) and textual (*e.g.*, natural language commands) information. The integration of multimodal inputs significantly enhances the decision making capability of GUI agents, enabling more reliable and consistent performance.

However, such improvements primarily stem from better GUI semantic understanding, while the generated GUI interactions still deviate substantially from human interaction patterns. Specifically, existing GUI agents still struggle to perform *swipe*, one of the most frequently used interactions. A key reason is that most GUI agents *implicitly assume* GUI interactions to be *component-centric*. They formulate GUI interaction as an action prediction task that maps a natural language command (*e.g.*, tap a button, type text into an input field) to an action type t and the coordinate (x, y) of a target GUI component. Some approaches [7, 9, 14, 33] even simplify this formulation as a pure GUI grounding task, *i.e.*, predicting only the target component's coordinate (x, y) .

Such overly simplified, component-centric interaction strategy *does not hold* for swipes. As shown in Figure 1, swipes in practice can be broadly categorized into two types: 1) *Swiping within a component* for fine-grained adjustments (*e.g.*, dragging the exposure slider in Lightroom¹), and 2) *Swiping over a general region* for content exploration (*e.g.*, swiping to reveal additional options in a lifestyle app, Meituan²). Unlike clicks and text inputs, swipes

*These authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License. MM '26, Rio de Janeiro, Brazil

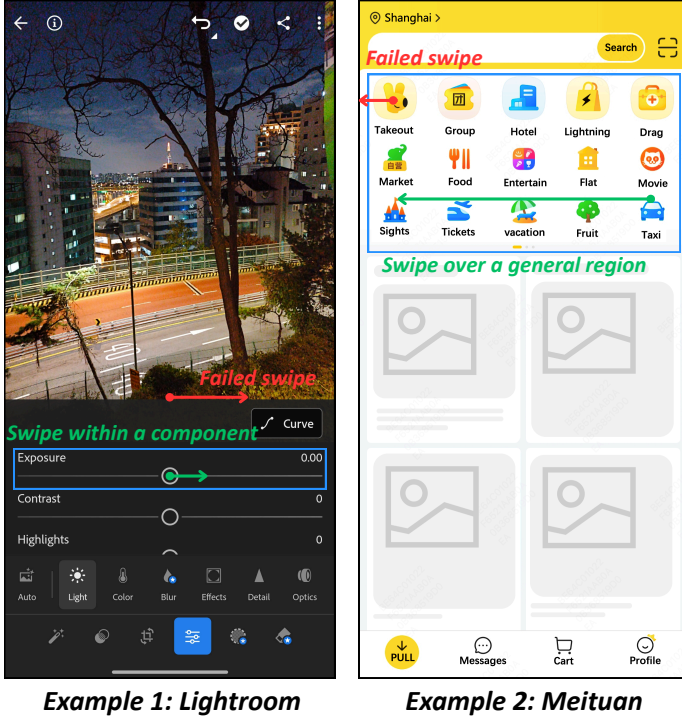
© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2213-4/2026/11

<https://doi.org/10.1145/3767308.3835803>

¹<https://play.google.com/store/apps/details?id=com.adobe.lrmobile&hl=en>

²<https://play.google.com/store/apps/details?id=com.sankuai.meituan&hl=en>



Example 1: Lightroom

Example 2: Meituan

Figure 1: Two common types of swipes in mobile GUIs.

are not necessarily anchored to specific components. As a result, existing VLM-based agents struggle to handle swipes under current task formulations, preventing them from completing a wide range of everyday tasks.

Addressing this issue requires overcoming two fundamental obstacles: 1) *Scarcity of annotated swipe data*. Existing GUI interaction datasets [5, 6, 8, 9, 19, 20, 24, 31, 39, 46] are heavily dominated by component-centric actions (e.g., clicks and text inputs), which account for 76.4% to 94.9% of all interactions, while swipes are sparsely annotated. 2) *Improper formulation of swipe*. Unlike clicks, a swipe requires accurately predicting multiple parameters, including the starting position, ending position, direction, and duration. However, the few datasets that do include swipe interactions [5, 6, 20, 24, 46] typically either ignore these parameters or oversimplify them (e.g., containing only a coarse direction). As a result, fine-tuned VLMs can hardly ground swipes correctly or generate valid swipe parameters when such interactions are required.

In this paper, we present SwipeGen, an automatic pipeline for synthesizing human-like swipe data without relying on predefined human commands. To properly formulate swipe interactions, SwipeGen decomposes each swipe into multiple execution dimensions, including the starting position, direction, distance, and velocity, consistent with widely used mobile automation tools [2, 11, 12]. Based on this definition, SwipeGen then automatically explores GUIs to synthesize human-like swipes and record all execution-required parameters. Specifically, it first detects scrollable targets (components and regions) on the screen, then executes candidate

Example 1: Lightroom, Swipe within a component

User Query: Adjust exposure to +0.20.

QWen2.5-VL: {"type": "swipe", "target": (540,1500), "direction": "right", "distance": 200}

Fail Reason: The swipe starts from *an incorrect coordinate* and uses *an inappropriate distance*, causing the interaction to miss the exposure slider.

Example 2: Meituan, Swipe over a general region

User Query: Swipe left across the icon area for more icons.

QWen2.5-VL: {"type": "swipe", "target": (120,320), "direction": "left", "distance": 120}

Fail Reason: The swipe starts from *an incorrect coordinate*, and the swipe *distance is too short to reveal* the next set of icons.

swipes, and finally verifies their validity by comparing GUI states before and after the interaction. Finally, SwipeGen retains only swipes that induce visual changes, enabling high-quality data collection.

Based on this pipeline, we construct and release SwipeBench, the first benchmark for evaluating the swipe execution capability of GUI agents. Furthermore, we demonstrate the effectiveness of SwipeGen by fine-tuning an open-source VLM for GUI interaction, resulting in a swipe-capable VLM, GUISwiper. GUISwiper achieves a swipe success rate of 61.25% on SwipeBench while maintaining competitive grounding performance. These results show that improving swipe capability does not come at the cost of degraded grounding performance, and validate the effectiveness of SwipeGen.

Our primary contributions are as follows:

- By comparing the success rates of swipe and other interactions, we are the first to pinpoint the widely-adopted rigid swipe execution strategy as a key bottleneck limiting the interaction capability of existing GUI agents.
- We design SwipeGen, the first automated pipeline for synthesizing human-like swipe interactions. Moreover, in order to advance research on swipe-aware GUI agents, we built SwipeBench, the first benchmark for evaluating swipe interaction quality.
- We prove that VLMs fine-tuned with SwipeGen-synthesized data can substantially address the swipe execution bottleneck in GUI agents. Experiments show that GUISwiper improves swipe success rate by 2.46×, shedding light on the development of more capable GUI agents.

Dataset	#Interactions	Action Distribution (%)			Swipe Annotation				
		Click+Text	Swipe	Other	Description	Start Pos	End Pos	Direction	Duration
AndroidHowTo [20]	136,023	94.9	5.1	10.9	✓	×	×	×	×
MoTIF [5]	12,244	94.1	5.9	0.0	×	✓	✓	✓	×
GUI-Odyssey [24]	110,056	89.4	9.5	6.9	×	✓	✓	×	×
CAGUI [46]	3,916	97.4	2.0	0.6	×	✓	✓	×	×
AMEX [6]	35,661	76.4	21.4	2.2	×	✓	✓	×	×

Table 1: Action distribution and swipe parameter annotation across representative GUI navigation datasets. Most existing datasets are dominated by component-centric actions (click and text input), while swipes are sparsely annotated. (Other denotes dataset-specific actions such as system-level back or composite gestures.)

2 Related Work

2.1 VLM for GUI Agents

Despite the strong capabilities of general-purpose large VLMs like GPT-4V [28], their performance in understanding and interacting with GUIs remains limited [42]. This limitation has motivated researchers to fine-tuning open-source VLMs, such as the Qwen-VL series [3, 4, 34, 37], to build GUI-specific models that better comprehend user commands and GUI images.

Training these models generally follows two paradigms: early-stage supervised fine-tuning (SFT) and more recent reinforcement learning (RL) approaches. Under the SFT paradigm, models such as SeeClick [9], CogAgent [15], OS-Atlas [39], UGround [14], ShowUI [21], and UI-TARS [30] demonstrate improved GUI understanding through supervised training on large, labeled datasets. However, SFT heavily depends on high-quality annotations, resulting in high training costs. With the emergence of DeepSeek-R1 [10], researchers found that reinforcement learning with verifiable rewards (RLVR), particularly group relative policy optimization (GRPO) [32], is well suited for GUI navigation tasks. Models such as UI-R1 [25], GUI-R1 [26], BTL-UI [45] and GUI-Owl [43] leverage predefined, task-specific reward functions to train more capable GUI-specific VLMs.

Overall, the performance of GUI-specific VLMs is strongly tied to the *distribution and quality of their training data*. This observation motivates our work, which targets the lack of diverse and executable swipe data by proposing a scalable pipeline to enhance the swipe capabilities of VLM for GUI agents.

2.2 Existing GUI Datasets

Existing GUI datasets can be broadly categorized into GUI grounding datasets and GUI navigation datasets. The GUI grounding task aims to locate the target GUI component given a natural language command. Under this component-centric formulation, GUI grounding datasets naturally emphasize click and text input actions, excluding swipes. Representative benchmarks such as ScreenSpot [9] and its subsequent variants [19, 39] follow this paradigm and therefore do not annotate swipes.

In contrast, GUI navigation datasets aim to model multi-step interaction trajectories for accomplishing high-level tasks. However, as summarized in Table 1, existing navigation datasets suffer from several limitations that hinder their use for training reliable swipes.

First, the *most critical limitation* is the lack of step-level natural language supervision. Most navigation datasets provide only a

single high-level task description (e.g., book a hotel near the city center) paired with a sequence of annotated low-level interactions. Individual actions, including swipes, are *not associated with corresponding descriptions*, making these datasets unsuitable for training single-step swipe prediction models. Second, the action distribution is heavily skewed toward component-centric interactions such as clicks and text inputs. Third, even when swipes are included, their annotations are often incomplete. As shown in Table 1, existing datasets typically miss one or more parameters required for executable swipes, such as explicit direction or duration. Although the swipe direction can be inferred from the start and end positions, we argue that explicitly annotating direction is important. Moreover, swipe duration directly affects the execution outcome due to OS-level gesture dynamics [41] (e.g., inertia). We would further discuss the role of these parameters in Section 3.1.

Overall, these limitations motivate the need for data that provides step-level language description and complete parameter annotations for swipes.

3 SwipeGen

This section introduces SwipeGen, an automatic pipeline for synthesizing executable swipe data without relying on predefined commands. Figure 2 provides an overview of the pipeline.

Given a GUI screen, SwipeGen first identifies scrollable targets (i.e., components and regions). Based on the identified targets, SwipeGen generates and executes candidate swipes under a unified swipe representation. To ensure data quality, each executed swipe is verified by comparing GUI images before and after it. For each valid swipe, SwipeGen generates a description based on the pre- and post-swipe images together with the swipe parameters.

3.1 Unified Swipe Representation

Before introducing SwipeGen, a fundamental question is unanswered: what parameters should a swipe annotation include for practical deployment?

For SwipeGen, our guiding principle is that synthesized swipe data should be *directly executable* in real-world GUI agent systems. In GUI agent systems, the outputs of VLMs are consumed by downstream GUI automation tools to interact with GUIs [36, 44]. Therefore, a valid swipe representation must contain sufficient parameters to invoke these tools.

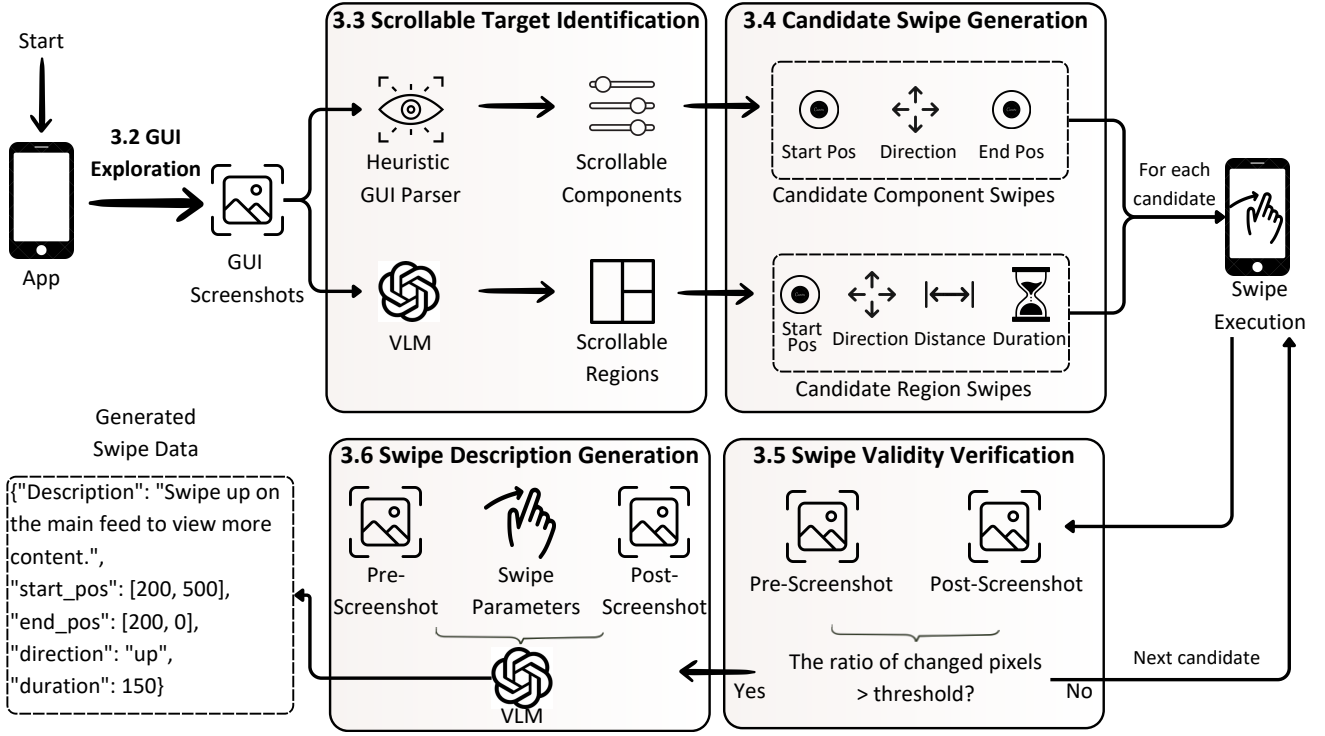


Figure 2: Overview of the proposed pipeline SwipeGen. It consists of five important modules: GUI Exploration Module, Scrollable Target Identification Module, Candidate Swipe Generation Module, Swipe Validity Verification Module and Swipe Description Generation Module.

To this end, we survey widely used mobile GUI automation tools for both Android and iOS, including Android Debug Bridge (ADB) [11], UIAutomator [12], and Appium [2]. And we found out that all these tools parameterize swipes using explicit start and end positions, and two of them (ADB and Appium) further support specifying the swipe duration.

We then analyze how these parameters affect different types of swipe interactions. For component-centric swipes that aim to make fine-grained adjustments (e.g., adjusting a slider), the execution outcome is determined by the start and end positions. In contrast, swipes over general regions exhibit different characteristics. As illustrated in Figure 1, the resulting scroll behavior is mainly effected by the starting position, the swipe direction, and the duration. Notably, duration directly controls the swipe velocity, which in turn determines how far the content scrolls. As a result, even with identical start and end positions, varied swipe duration can lead to different effect. And the exact endpoint is less important for region-level swipes. Therefore, although the direction can be derived from the start and end positions, we explicitly annotate it instead to facilitate learning controllable swipes.

Based on these observations, we unify swipes using four explicit parameters: start position, end position, direction, and duration.

3.2 GUI Exploration

To collect diverse GUI screens for subsequent swipe data generation, SwipeGen adopts a depth-first search (DFS)-based exploration

strategy. Starting from the app’s entry point, it systematically navigates the GUI by interacting with clickable components. For each visited screen, SwipeGen records the sequence of interactions required to reach it, enabling efficient revisiting for later scrollable target identification.

For each screen, SwipeGen identifies unvisited clickable components based on the underlying GUI hierarchy files (e.g., XML layouts [1]) and executes interactions to discover new screens. For each visited screen, SwipeGen records the interaction path for efficient revisiting.

To distinguish newly discovered screens from previously visited ones, we represent each screen using a lightweight skeleton abstraction that reduces the influence of dynamic content. SwipeGen compares the skeleton similarity between screens and continues exploration until no new screens are discovered.

3.3 Scrollable Target Identification

For each collected GUI screen, SwipeGen identifies two types of scrollable targets: scrollable components and scrollable regions.

Scrollable components are explicit GUI elements that support swipe interactions, such as sliders and progress bars. These components can be directly identified via heuristic parsing of XML layouts. If a node’s scrollable attribute is set to true or its class includes RecyclerView, ScrollView or SeekBar, its corresponding component is regarded as scrollable.

Task:

You are given a screenshot of a mobile app UI. Your task is to identify all scrollable regions on the screen, *i.e.*, areas that support horizontal or vertical swipes, such as content feeds, lists, carousels, or grid layouts.

Output:

For each region, provide the following information:

1) Type: a brief description of the region (*e.g.*, list, feed, carousel). 2) Direction: if scrollable, the supported swipe direction ("horizontal", "vertical", or "both"). 3) Bbox: bounding box coordinates $[x1, y1, x2, y2]$, where x and y range from 0 to 1000. 4) Description: a short description of the intended interaction. Output the result in JSON format. Only output valid JSON. Do not include any additional text.

Example:

```
[{
  "Type": "content feed",
  "Direction": "vertical",
  "Bbox": [0, 0, 1000, 1000],
  "Description": "Swipe up or down to view
  more content in the feed"
}, ... ]
```

Figure 3: The prompt for identifying scrollable regions.

Scrollable regions, in contrast, refer to layout-level areas that support exploratory swipes, such as content feeds, lists, and icon grids. Unlike scrollable components, these regions often do not correspond to explicit nodes in GUI hierarchy files. Instead, they are defined by their visual layout and semantic function. We therefore leverage a VLM to directly infer scrollable regions from GUI screenshots. Specifically, we use Qwen3-VL-4B-Instruct [34], an advanced multimodal model with strong GUI understanding capability, to reason over visual appearance. The prompt design is shown in Figure 3.

3.4 Candidate Swipe Generation

Given the identified scrollable targets, SwipeGen generates candidates according to the target type.

3.4.1 Scrollable Components. For scrollable components, we generate candidates as follows. Let $b = (x_1, y_1, x_2, y_2)$ denote the component bounding box. We set the swipe start position to the box center,

$$s = (s_1, s_2) = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right) \quad (1)$$

The swipe orientation is determined by the component aspect ratio: vertical swipes are considered if $(y_2 - y_1) > (x_2 - x_1)$, and horizontal swipes otherwise. A swipe direction dir is then randomly sampled from the valid orientations (up and down for vertical swipes, left and right for horizontal swipes). The swipe distance is randomly sampled as a fraction of the screen size.

Specifically, let $\alpha \in (0, 1]$ denote a random scaling factor. For a horizontal(vertical) swipe, the distance is defined as

$$d = \alpha \cdot L, \quad (2)$$

where L is the screen width or height. Given the sampled direction, the end position is obtained by translating the start position s along the swipe direction by distance d , while ensuring the end position remains within the screen boundary. For a horizontal swipe, the end position is computed as

$$e = \begin{cases} (\min(s_1 + d, W), s_2), & \text{if direction is right,} \\ (\max(s_1 - d, 0), s_2), & \text{if direction is left,} \end{cases} \quad (3)$$

and for a vertical swipe,

$$e = \begin{cases} (s_1, \min(s_2 + d, H)), & \text{if direction is down,} \\ (s_1, \max(s_2 - d, 0)), & \text{if direction is up,} \end{cases} \quad (4)$$

As discussed in Section 3.1, duration plays a limited role for such swipes. Therefore, we fix the duration t to a default value of 300ms. Overall, each component yields 2 candidate swipes with different swipe directions, represented as (s, e, dir) .

3.4.2 Scrollable Regions. Similarly, for each scrollable region with bounding box b , we first compute its center position c . We identify the dominant axis of the region by comparing its height and width. If $(y_2 - y_1) > (x_2 - x_1)$, the region is treated as vertically scrollable. Otherwise, it is treated as horizontally scrollable. Then, we offset the center c along the dominant axis to generate candidate start positions close to the region's boundary. Specifically, let $\alpha \in [0.2, 0.5]$ denote a random offset ratio. For a horizontally scrollable region, the candidate start positions are

$$s = \begin{cases} (c_1 + \alpha(x_2 - x_1), c_2), \\ (c_1 - \alpha(x_2 - x_1), c_2), \end{cases} \quad (5)$$

and for a vertically scrollable region,

$$s = \begin{cases} (c_1, c_2 + \alpha(y_2 - y_1)), \\ (c_1, c_2 - \alpha(y_2 - y_1)). \end{cases} \quad (6)$$

For each start position s , the swipe direction is set opposite to the offset direction, reflecting natural scrolling behavior (*e.g.*, offsetting right corresponds to a left swipe). The end position is then obtained by extending the swipe along this direction until reaching the region boundary. For horizontal swipes,

$$e = \begin{cases} (x_1, s_2), & \text{if direction is left,} \\ (x_2, s_2), & \text{if direction is right,} \end{cases} \quad (7)$$

and for vertical swipes,

$$e = \begin{cases} (s_1, y_1), & \text{if direction is up,} \\ (s_1, y_2), & \text{if direction is down.} \end{cases} \quad (8)$$

Moreover, the execution outcome of swipes over a region is sensitive to swipe speed. In real-world systems, user control over swipe gestures is inherently coarse-grained (*e.g.*, fast vs. slow swipes), rather than continuous. This is also reflected in OS-level gesture recognizers, which typically rely on velocity thresholds to determine scrolling behavior. Accordingly, we divide swipe duration t into two categories: a fast swipe (150ms) and a slow swipe (500ms). Overall, each region yields $2 \times 2 = 4$ candidate swipes with different start points and duration, represented as (s, e, dir, t) .

Variant	Spotify	DeepSeek	Twitter/X	Overall
w/o Verification	74	14	87	175
SwipeGen	54 (↓ 27%)	4 (↓ 71%)	70 (↓ 20%)	128 (↓ 27%)

Table 2: Ablation of SwipeGen.

Task:

You are given two screenshots of a mobile app UI, the screen before a swipe and the screen after it. You are also given the executed swipe with its parameters. Your task is to generate a concise, step-level natural language command written in imperative form that accurately describes the performed swipe.

Output:

The generated command should: 1. Describe a single interaction step. 2. Based on the visual change between the two screenshots. 3. Reflect the intent of the swipe (e.g., revealing more content, scrolling a list). Output the result in JSON format, include the command itself ("Command") and the reason why you describe it as so ("Reason").

Example:

```
{
  "Command": "Swipe up on the main feed to view more content.",
  "Reason": ...
}
```

Figure 4: The prompt for generating swipe descriptions.

3.5 Swipe Validity Verification

However, not every candidate swipe leads to a GUI response. To ensure data quality, SwipeGen adopts an execute-and-verify procedure for swipe selection. For each scrollable target, candidate swipes are executed sequentially and verified one by one, and the first swipe that induces a perceptible GUI change is retained as a valid sample.

To achieve this, we compare the visual changes between the screenshots before and after execution. Specifically, we convert the two screenshots to grayscale, and restrict the comparison within the target area (i.e., the bounding box of the scrollable component or region). We then compute the pixel-wise absolute difference and measure the ratio of pixels whose intensity change exceeds a fixed threshold $\delta = 0.02$. If the swipe is considered effective and retained, SwipeGen moves on to the next GUI. Otherwise, the current candidate is discarded, and SwipeGen executes the next swipe.

Moreover, to understand the contribution of this module, we conduct an ablation study by removing this module while keeping the remaining pipeline unchanged. The results are shown in table 2.

3.6 Swipe Description Generation

Finally, following the reverse-synthesis practice [18, 40], for each validated swipe, SwipeGen generates a corresponding step-level natural language description. Specifically, we prompt a VLM with

the GUI screenshots before and after the swipe, together with the executed swipe parameters, and ask it to describe the performed interaction in natural language. The prompt and example outputs are provided in Figure 4.

4 SwipeBench

In order to further advance research on swipe-aware GUI agents, we introduce SwipeBench. It is the first benchmark specifically designed to evaluate swipe execution in GUI agents. SwipeBench focuses on assessing the performance of GUI agents in out-of-domain (OOD) scenarios, thereby minimizing potential data leakage from VLM pretraining. Specifically, all the selected apps in SwipeBench are newly released, ensuring no overlap with commonly used or previously benchmarked mobile apps, which are often included in large-scale vision-language pretraining corpora.

SwipeGen initially generated 421 swipe interactions from 16 distinct mobile apps, exploring key screens reachable from the home page through main navigation tabs. After manual filtering to remove instances containing sensitive user information (e.g., personal profiles) and swipes without page responses, SwipeBench comprises 382 valid interactions. Each swipe data is multimodal, including images before and after the interaction, a natural language description of the interaction, the type of interaction, and any necessary parameters for the interaction. The distribution of these swipes across different applications can be found at our GitHub repository.

5 Evaluation

This section evaluates the data generation capability of SwipeGen and the effectiveness of its generated data using GUISwiper.

5.1 GUISwiper

We fine-tuned GUISwiper, a GUI-specific VLM capable of performing multiple types of interactions with GUIs, particularly swipes. To enhance generalization while keeping training costs low, we employed the advanced RLVR method [7, 22, 25, 26, 33]. Additionally, to assess the quality of the data generated by SwipeGen, we selected one of the most widely used base models, Qwen2.5-VL-3B-Instruct [4].

5.1.1 Training Dataset. The training dataset consists of 927 swipe interactions and 579 tap interactions, collected from 43 popular applications. The apps were selected based on their ranking within Google Play categories, ensuring that the most popular app in each category was included. The swipe data was automatically generated using SwipeGen, while the tap data was collected during the GUI exploration phase and later annotated by a VLM.

5.1.2 Runtime Analysis. We additionally analyze the computational cost of SwipeGen. Table 3 reports the runtime and token consumption of each module when generating the GUISwiper training set.

5.1.3 Reward Design. We design the overall reward as a combination of three components: a format reward, an action type reward, and an action accuracy reward. The final reward is linearly normalized to $[-1, 1]$ for stable training.

Module	Avg. Time	Prompt Token	Completion Token
GUI Exploration	38.82 s / page	–	–
Scrollable Target Identification	13.90 s / page	2860 / page	67 / page
Swipe Validity Verification	4.34 s / swipe	–	–
Swipe Description Generation	13.47 s / swipe	5668 / swipe	57 / swipe
Total	120.46 s / swipe	9374 / swipe	144 / swipe

Table 3: Runtime and token cost of SwipeGen.

Model	Model Size	Success Rate (%)
Qwen2.5-VL [4]	3B	24.87
Qwen3-VL [34]	2B	39.79
UGround [14]	2B	18.85
UI-R1 [25]	3B	30.41
GUI-Owl [43]	2B	30.37
UI-TARS [30]	2B	52.09
MAI-UI [47]	2B	48.95
GUISwiper	3B	61.25

Table 4: Swipe execution success rate on SwipeBench.

We adopt a widely used format reward [25, 45] to enforce structured outputs. The model is required to produce reasoning enclosed in `<think>` tags followed by a final answer in a predefined JSON format. Correctly formatted outputs receive a reward of +1, while malformed outputs receive −1.

To prevent overfitting to swipes and preserve general GUI interaction capabilities, we include an action type reward. If the predicted action type (e.g., swipe, click, input) matches the ground truth, the model receives +0.8; otherwise, −0.8.

For non-swipe actions, we follow the same accuracy reward design as UI-R1 [25]. As for swipes, we define an accuracy reward $R_{acc} \in [0, 1]$ as the sum of four sub-rewards:

$$R_{acc} = R_{start} + R_{end} + R_{dir} + R_{dur}. \quad (9)$$

(1) If the Euclidean distance between the predicted and ground-truth start positions is within 220 pixels, a reward of 0.45 is given; otherwise, 0. Following prior work [22], we normalize all the image resolutions to (0, 0, 1000, 1000), and set a 220-pixel tolerance for allowing reasonable localization errors. Additionally, for region-level swipes, a constraint is enforced: if the predicted start position lies outside the target bounding box, the start-position reward is set to 0 regardless of distance.

(2) If the predicted end position is within 220 pixels of the ground truth, a reward of 0.10 is given. This term is assigned a lower weight since end positions are less important for scrollable regions.

(3) If the predicted swipe direction (e.g., up) matches the ground truth, a reward of 0.35 is given.

(4) For component-level swipes, the reward of 0.1 is always granted. For region-level swipes, we discretize duration into fast (150 ms) and slow (500 ms). Predicted durations are mapped to these categories using a midpoint threshold, and a reward of 0.10 is given if the predicted category matches the ground truth.

Notably, all accuracy rewards are non-negative: incorrect predictions do not incur penalties, which is crucial for stable training.

5.2 Experimental Settings

5.2.1 Benchmarks. We evaluate GUI agents on three benchmarks: SwipeBench, ScreenSpot [9] and AndroidWorld [31].

ScreenSpot is a widely used GUI grounding evaluation benchmark. We evaluate GUISwiper on its mobile subset.

AndroidWorld is an online benchmark consisting of 116 multi-step GUI automation tasks. Unlike offline grounding benchmarks, AndroidWorld requires agents to interact with dynamic environments through sequential decision-making and action execution.

5.2.2 Baselines. We compare GUISwiper against multiple state-of-the-art (SOTA) GUI-specific VLMs.

- Qwen2.5-VL-3B-Instruct [4], the foundation model.
- Qwen3-VL-2B-Instruct [34], the latest one in the Qwen series. It is specifically trained on GUI data.
- UGround [14] is trained via supervised fine-tuning (SFT) on a dataset comprising 10M GUI elements and over 1.3M screenshots.
- UI-R1 [25] and GUI-Owl [43] employ RL-based training, representing recent advances in this paradigm.
- UI-TARS [30] and MAI-UI [47] are industrial SOTA models trained on large-scale proprietary GUI data.

5.3 Experimental Result and Analysis

5.3.1 Swipe Execution. Table 4 reports the swipe execution success rate on SwipeBench. We formulate swipe execution as a binary classification task, where a swipe is considered successful only if all the parameters satisfy the same criteria used during training.

As shown in the table, directly prompting a Qwen2.5-VL yields a low success rate of 24.87%. In contrast, GUISwiper improves the success rate to 61.25%, achieving a 2.46× relative improvement and outperforming all baselines, including UI-TARS (52.09%) and MAI-UI (48.95%).

Furthermore, GUISwiper successfully completes 23 tasks where all baseline models fail. These tasks mainly involve long-range scrolling, feed navigation, and navigation exploration, highlighting the importance of reliable swipe execution for complex GUI interactions.

Moreover, we analyze the failure cases of GUISwiper to understand the remaining challenges. Region-level errors are the dominant failure source (69%), where the model predicts swipe locations outside valid interaction regions. Distance estimation errors account for another 25% of failures, especially when the instruction does not explicitly specify swipe magnitude. In contrast, direction errors only account for 6%, suggesting that coarse directional understanding is no longer the primary bottleneck.

Model	Model Size	Text Accuracy (%)	Icon/Widget Accuracy (%)	Avg. (%)
Qwen2.5-VL [4]	3B	90.50	61.10	75.80
UGround [14]	2B	82.80	60.30	71.60
UI-R1 [25]	3B	95.60	84.70	90.15
UI-TARS [30]	2B	93.00	75.50	84.25
GUISwiper	3B	<u>91.20</u>	<u>80.32</u>	<u>85.83</u>

Table 5: Grounding accuracy on ScreenSpot.

Model	Model Size	Success Rate (%)
<i>Comparable-scale Models</i>		
Qwen3-VL	2B	36.40
UGround	2B	32.80
MAI-UI	2B	49.10
GUI-Owl	2B	73.30
<i>Large-scale Models</i>		
UI-TARS	7B	33.00
MAI-UI	8B	70.70
Qwen3-VL	8B	47.60
Qwen3-VL	32B	57.30
Qwen2.5-VL	32B	22.00
Qwen2.5-VL	72B	35.00
UI-TARS	72B	46.60
GUISwiper	3B	40.52

Table 6: End-to-end task success rate on AndroidWorld.

5.3.2 Grounding Capability. Table 5 reports grounding accuracy on ScreenSpot. Our primary goal is to examine whether fine-tuning on interaction data degrades general GUI grounding capability. Some recent models (e.g., Qwen3-VL, GUI-Owl, MAI-UI) do not report results on ScreenSpot, and are therefore excluded from this comparison.

As shown in the table, GUISwiper achieves an overall accuracy of 85.83%, outperforming its backbone model Qwen2.5-VL (75.80%) while remaining competitive with strong baselines such as UI-TARS (84.25%) and UI-R1 (90.15%). These results suggest that enhancing region-level swipe capabilities does not come at the cost of degraded element localization. Overall, the results demonstrate that swipe execution and grounding capability can be jointly maintained without trade-offs.

5.3.3 End-to-end Task Execution. Table 6 reports end-to-end task success rates on AndroidWorld. GUISwiper achieves a success rate of 40.52%, outperforming comparable-scale models such as Qwen3-VL-2B (36.40%) and UGround (32.80%), as well as larger backbone models including Qwen2.5-VL-32B (22.00%) and Qwen2.5-VL-72B (35.00%). These results demonstrate that improving swipe execution can translate into better downstream task completion. Note that GUI-Owl achieves higher performance through the MobileAgent-v3 workflow with additional agentic components, while GUISwiper focuses only on low-level swipe interaction.

We further analyze GUISwiper’s execution traces and find that successful tasks often involve long-horizon off-screen navigation. For example, in RecipeAddMultipleRecipes, the agent repeatedly

scrolls through long recipe pages while continuously entering content into the Broccoli app, with scroll actions accounting for 41.9% of all executed steps. Similar tasks include RecipeAddSingleRecipe, ExpenseAddMultiple, etc. These cases show that reliable swipe execution is important for maintaining progress when agents repeatedly access invisible UI content.

Overall, these results indicate that swipe execution is an important bottleneck for end-to-end GUI automation.

6 Discussion

6.1 Limitations

While SwipeGen improves GUI agents’ swipe execution capability, we acknowledge several limitations. First, our definition of human-like focuses on the task-effect level, *i.e.*, whether a swipe aligns with user intent and induces the expected GUI transition. It does not aim to reproduce the complete motor characteristics of human swipes, such as trajectory curvature or velocity variation.

Second, our Swipe Validity Verification module adopts light-weight pixel-level comparison to enable scalable data synthesis. Although we restrict the comparison area to the target scrollable component or region, dynamic content within this area may still introduce false positives.

6.2 Future Work

We identify several promising directions for future work. First, we plan to investigate more fine-grained human swipe modeling by incorporating real user interaction traces, including trajectory patterns and temporal dynamics. Such modeling could further narrow the gap between GUI agents and human users.

Second, we plan to scale up both synthesized training data and evaluation benchmarks to cover more diverse applications and long-horizon interaction scenarios.

7 Conclusion

In this work, we address a critical yet overlooked limitation in GUI agents: their inability to execute swipe interactions effectively due to prevailing component-centric interaction strategies.

Our contributions are three-fold. First, we propose SwipeGen, the first automated pipeline for synthesizing human-like swipe interactions, enabling proper formulation of swipe data that has been lacking in existing datasets. Second, we introduce SwipeBench, the first benchmark for evaluating swipe interaction quality, comprising 382 swipes from 16 OOD mobile apps. Third, through GUISwiper, we show that VLMs fine-tuned with synthesized swipe data can substantially address the swipe execution bottleneck.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Project No. 62572127). Y.Zhou is the corresponding author.

References

- [1] 2026. *Layouts in Views*. <https://developer.android.com/develop/ui/views/layout/declaring-layout>
- [2] Appium. 2025. Appium: Mobile Automation Framework. <https://github.com/appium/appium>
- [3] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. *arXiv preprint arXiv:2308.12966* (2023).
- [4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).
- [5] Andrea Burns, Deniz Arsan, Sanjna Agrawal, Ranjitha Kumar, Kate Saenko, and Bryan A. Plummer. 2022. A Dataset for Interactive Vision-Language Navigation with Unknown Command Feasibility. In *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part VIII (Lecture Notes in Computer Science, Vol. 13668)*, Shai Avidan, Gabriel J. Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner (Eds.). Springer, 312–328.
- [6] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Guozhi Wang, Dingyu Zhang, Shuai Ren, and Hongsheng Li. 2025. AMEX: Android Multi-annotation Expo Dataset for Mobile GUI Agents. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 2138–2156.
- [7] Liangyu Chen, Hanzhang Zhou, Chenglin Cai, Jianan Zhang, Panrong Tong, Quyu Kong, Xu Zhang, Chen Liu, Yuqi Liu, Wenxuan Wang, Yue Wang, Qin Jin, and Steven Hoi. 2025. UI-Ins: Enhancing GUI Grounding with Multi-Perspective Instruction-as-Reasoning. *CoRR abs/2510.20286* (2025).
- [8] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, Yuan Yao, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2025. GUICourse: From General Vision Language Model to Versatile GUI Agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 21936–21959.
- [9] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 9313–9332.
- [10] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR abs/2501.12948* (2025).
- [11] Android Developers. 2025. Android Debug Bridge (adb). <https://developer.android.com/tools/adb>
- [12] Android Developers. 2025. Write automated tests with UI Automator. <https://developer.android.com/training/testing/other-components/ui-automator>
- [13] Sidong Feng, Changhao Du, Huaxiao Liu, Qingnan Wang, Zhengwei Lv, Gang Huo, Xu Yang, and Chunyang Chen. 2025. Agent for User: Testing Multi - User Interactive Features in TikTok. In *47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2025, Ottawa, ON, Canada, April 27 - May 3, 2025*, IEEE, 57–68.
- [14] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*, OpenReview.net.
- [15] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazhen Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, IEEE, 14281–14290.
- [16] Yongxiang Hu, Xuan Wang, Yingchuan Wang, Yu Zhang, Shiyu Guo, Chaoyi Chen, Xin Wang, and Yangfan Zhou. 2024. AUTestAgent: Automatic Requirements Oriented GUI Function Testing. *CoRR abs/2407.09018* (2024).
- [17] Satwik Ram Kodandaram, Utku Uckun, Xiaojun Bi, I. V. Ramakrishnan, and Vikas Ashok. 2024. Enabling Uniform Computer Interaction Experience for Blind Users through Large Language Models. In *The 26th International ACM SIGACCESS Conference on Computers and Accessibility, ASSETS 2024, St. John's, NL, Canada, October 27-30, 2024*, David R. Flatla, Faustina Hwang, Tiago João Vieira Guerreiro, and Robin Brewer (Eds.). ACM, 73:1–73:14.
- [18] Minkyu Kweon, Seokhyeon Park, Soohyun Lee, You Been Lee, Jeongmin Rhee, and Jinwook Seo. 2026. GhostUI: Unveiling Hidden Interactions in Mobile UI. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, CHI 2026*.
- [19] Kaixin Li, Meng Ziyang, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. 2025. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. In *Workshop on Reasoning and Planning for Large Language Models*.
- [20] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. 2020. Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Annual Conference of the Association for Computational Linguistics (ACL 2020)*.
- [21] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*, Computer Vision Foundation / IEEE, 19498–19508.
- [22] Yuhang Liu, Zeyu Liu, Shuanghe Zhu, Pengxiang Li, Congkai Xie, Jiasheng Wang, Xueyu Hu, Xiaotian Han, Jianbo Yuan, Xinyao Zhang, Shengyu Zhang, Hongxia Yang, and Fei Wu. 2025. InfiGUI-G1: Advancing GUI Grounding with Adaptive Exploration Policy Optimization. *CoRR abs/2508.05731* (2025).
- [23] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Mengzhou Chen, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. 2025. Seeing is Believing: Vision-Driven Non-Crash Functional Bug Detection for Mobile Apps. *IEEE Trans. Software Eng.* 51, 12 (2025), 3452–3466.
- [24] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2024. GUI Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. *CoRR abs/2406.08451* (2024).
- [25] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *CoRR abs/2503.21620* (2025).
- [26] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. 2025. GUI-R1 : A Generalist R1-Style Vision-Language Action Model For GUI Agents. *CoRR abs/2504.10458* (2025).
- [27] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Md. Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Jihyung Kil, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. 2025. GUI Agents: A Survey. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 22522–22538.
- [28] OpenAI. 2023. GPT-4V(ision) System Card. https://cdn.openai.com/papers/GPT_TV_System_Card.pdf
- [29] Yi-Hao Peng, Dingzeyu Li, Jeffrey P. Bigham, and Amy Pavel. 2025. Morae: Proactively Pausing UI Agents for User Choices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology, UIST 2025, Busan, Korea, 28 September 2025 - 1 October 2025*, Andrea Bianchi, Elena L. Glassman, Wendy E. Mackay, Shengdong Zhao, Jeeun Kim, and Ian Oakley (Eds.). ACM, 198:1–198:14.
- [30] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *CoRR abs/2501.12326* (2025).
- [31] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E. Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P. Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*, OpenReview.net.
- [32] Zhihong Shao, Peiyi Wang, Qihao Zhang, Runxin Xu, Junxia Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR abs/2402.03300* (2024).
- [33] Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and

- Yueting Zhuang. 2025. GUI-G²: Gaussian Reward Modeling for GUI Grounding. *CoRR* abs/2507.15846 (2025).
- [34] Qwen Team. 2025. Qwen3 Technical Report.
- [35] Minh Duc Vu, Han Wang, Jieshan Chen, Zhuang Li, Shengdong Zhao, Zhenchang Xing, and Chunyang Chen. 2024. GPTVoiceTasker: Advancing Multi-step Mobile Task Efficiency Through Dynamic Interface Exploration and Learning. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, UIST 2024, Pittsburgh, PA, USA, October 13–16, 2024*, Lining Yao, Mayank Goel, Alexandra Ion, and Pedro Lopes (Eds.). ACM, 48:1–48:17.
- [36] Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-Agent-v2: Mobile Device Operation Assistant with Effective Navigation via Multi-Agent Collaboration. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 – 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.).
- [37] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. *arXiv preprint arXiv:2409.12191* (2024).
- [38] Xuan Wang, Yingchuan Wang, Yongxiang Hu, Yu Zhang, Hailiang Jin, Shiyu Guo, Juxing Yuan, and Yangfan Zhou. 2025. From Redundancy to Efficiency: Exploiting Shared UI Interactions towards Efficient LLM-Based Testing. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16–20, 2025*. IEEE, 3438–3449.
- [39] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2025. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net.
- [40] Bin Xie, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Jie Liu, Min Zhang, and Liqiang Nie. 2025. GUI-explorer: Autonomous Exploration and Mining of Transition-aware Knowledge for GUI Agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, *ACL 2025*.
- [41] Shota Yamanaka, Hiroki Usuba, and Junichi Sato. 2024. Behavioral Differences between Tap and Swipe: Observations on Time, Error, Touch-point Distribution, and Trajectory for Tap-and-swipe Enabled Targets. In *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11–16, 2024*, Florian 'Floyd' Mueller, Penny Kyburz, Julie R. Williamson, Corina Sas, Max L. Wilson, Phoebe O. Toups Dugas, and Irina Shklovski (Eds.). ACM, 549:1–549:12.
- [42] An Yan, Zhengyuan Yang, Wanrong Zhu, Kevin Lin, Linjie Li, Jianfeng Wang, Jianwei Yang, Yiwu Zhong, Julian McAuley, Jianfeng Gao, Zicheng Liu, and Lijuan Wang. 2023. GPT-4V in Wonderland: Large Multimodal Models for Zero-Shot Smartphone GUI Navigation. *CoRR* abs/2311.07562 (2023).
- [43] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. 2025. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *arXiv:2508.15144 [cs.AI]*
- [44] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI 2025, Yokohama Japan, 26 April 2025– 1 May 2025*, Naomi Yamashita, Vanessa Evers, Koji Yatani, Sharon Xianghua Ding, Bongshin Lee, Marshini Chetty, and Phoebe O. Toups Dugas (Eds.). ACM, 70:1–70:20.
- [45] Shaojie Zhang, Ruoceng Zhang, Pei Fu, Shaokang Wang, Jiahui Yang, Xin Du, Shiqi Cui, Bin Qin, Ying Huang, Zhenbo Luo, and Jian Luan. 2025. BTL-UI: Blink-Think-Link Reasoning Model for GUI Agent. *CoRR* abs/2509.15566 (2025).
- [46] Zhong Zhang, Yaxi Lu, Yikun Fu, Yupeng Huo, Shenzhi Yang, Yesai Wu, Han Si, Xin Cong, Haotian Chen, Yankai Lin, Jie Xie, Wei Zhou, Wang Xu, Yuanheng Zhang, Zhou Su, Zhongwu Zhai, Xiaoming Liu, Yudong Mei, Jianming Xu, Hongyan Tian, Chongyi Wang, Chi Chen, Yuan Yao, Zhiyuan Liu, and Maosong Sun. 2025. AgentCPM-GUI: Building Mobile-Use Agents with Reinforcement Fine-Tuning. *arXiv preprint arXiv:2506.01391* (2025).
- [47] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. 2025. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents.